Module 3

🧪 Test Strategy: Phone Number Validation & Formatting Function

Role: Test Architect

As the test architect, my responsibility is to:

- Decide what behaviour must be tested
- Define what “correct” output looks like
- Identify risks, edge cases, and invalid inputs
- Evaluate whether testing is sufficient

AI assists with implementation details, but the testing strategy, specifications, and verification decisions are designed by me.

Specification

The function must validate and format phone numbers into the standard format:

```
(XXX) XXX-XXXX
```

The function should:

- Accept phone numbers written in different formats
- Allow spaces, dashes, and brackets
- Extract and validate numeric digits
- Reject invalid inputs
- Return a clear error message for invalid cases

Expected invalid output:

```
Error: Invalid phone number
```

---

1. Plain Language Test Scenarios

Happy Path Testing

These tests verify the normal expected behaviour:

- A standard 10-digit phone number should format correctly
- Numbers containing dashes should still work
- Numbers containing spaces should still work
- Numbers containing brackets around the area code should still work

Boundary Testing

These tests check the minimum and maximum valid limits:

- Exactly 10 digits should be accepted
- Fewer than 10 digits should be rejected
- More than 10 digits should be rejected

Invalid Input Testing

These tests verify that incorrect input is safely rejected:

- Inputs containing letters should fail
- Inputs containing only symbols should fail
- Empty input should fail
- Mixed letters and numbers should fail

Edge Case Testing

These tests verify unusual but possible scenarios:

- Leading and trailing spaces should still be handled correctly
- Different valid input styles should produce the same standardized output

---

2. Example Input / Output Specifications

Input	Expected Output
1234567890	(123) 456-7890
123-456-7890	(123) 456-7890
(123) 456 7890	(123) 456-7890
123 456 7890	(123) 456-7890

Input	Expected Output
`1234567890`	`(123) 456-7890`
`12345`	`Error: Invalid phone number`
`123456789012`	`Error: Invalid phone number`
`abc-def-ghij`	`Error: Invalid phone number`
(empty input)	`Error: Invalid phone number`

These examples clearly define what correct behaviour looks like before implementation begins.

---

3. Edge Cases and Boundaries

The following edge cases were identified during strategic test design:

- Exactly 10 digits → valid
- 9 digits → invalid
- More than 10 digits → invalid
- Empty input → invalid
- Input with symbols only → invalid
- Mixed letters and numbers → invalid
- Leading/trailing spaces → valid after cleaning

These cases help ensure the function behaves reliably under both normal and unusual conditions.

---

4. Invalid Input Handling

The function must reject:

- Alphabetic characters
- Invalid symbols
- Incorrect digit counts
- Empty or null input
- Inputs without exactly 10 digits

Expected error response:

```
Error: Invalid phone number
```

This ensures incorrect data is handled consistently and safely.

---

5. Executable Verification Tests (Assertions)

AI assisted with implementation details, while the test strategy and expected behaviour were defined beforehand.

```
def format_phone_number(number):
    import re

    # Reject alphabetic characters
    if re.search(r'[A-Za-z]', number):
        return "Error: Invalid phone number"

    # Extract digits only
    digits = re.sub(r'\D', '', number)

    # Validate length
    if len(digits) != 10:
        return "Error: Invalid phone number"

    # Return standardized format
    return f"({digits[:3]}) {digits[3:6]}-{digits[6:]}"


# Happy path tests
assert format_phone_number("1234567890") == "(123) 456-7890"
assert format_phone_number("123-456-7890") == "(123) 456-7890"
assert format_phone_number("(123) 456 7890") == "(123) 456-7890"

# Edge case test
assert format_phone_number(" 1234567890 ") == "(123) 456-7890"

# Invalid input tests
assert format_phone_number("12345") == "Error: Invalid phone number"
assert format_phone_number("123456789012") == "Error: Invalid phone number"
assert format_phone_number("abc-def-ghij") == "Error: Invalid phone number"
assert format_phone_number("") == "Error: Invalid phone number"
```

Evaluation of Test Adequacy

The tests are considered adequate because they cover:

- Happy path behaviour
- Boundary conditions
- Edge cases
- Invalid inputs
- Formatting consistency
- Error handling

The testing strategy focuses on behaviour rather than implementation details, providing stronger confidence that the function works correctly under realistic conditions.

Integrated Testing Cycle

Step 1 – Strategic Test Design

Define behaviors, risks, and expected outputs before implementation.

Step 2 – Implementation

Use AI assistance to generate the formatting and validation logic.

Step 3 – Verification

Run assertions to verify outputs match specifications.

Step 4 – Debugging

Investigate failures related to:

- Input cleaning
- Digit extraction
- Validation rules
- Formatting logic

Step 5 – Refinement

Correct issues and improve reliability through repeated testing.

Step 6 – Confidence Building

A successful solution:

- Passes all defined tests
- Handles edge cases correctly
- Rejects invalid input consistently
- Produces reliable standardized formatting

The End !!!